

Web Application Vulnerability Assessment Report

OWASP Juice Shop — Injection, Access Control & Client-Side Attacks

Target: <https://juice-shop.herokuapp.com/#/>

Researcher: **Abhimanyu Rawat**

Date: May 18, 2026

Role: Technical Research & Cyber Security Operations Intern

GreySentinel

Confidential – For Internal Training Use Only

1. Objective

This report documents hands-on security testing performed against the OWASP Juice Shop intentionally vulnerable web application. The objective was to identify, demonstrate, and document the following vulnerability categories: Injection Attacks, Broken Access Control & Authentication, and Client-Side Vulnerabilities. All activities were performed in a controlled training environment for educational purposes only.

2. Scope and Environment

Field	Value
Target Application	OWASP Juice Shop
Target URL	https://juice-shop.herokuapp.com/#/
Target IP	54.220.192.176 / 54.73.53.134
Testing Environment	Kali Linux
Tools Used	Burp Suite Community Edition, Browser DevTools, curl, Kali terminal
Date of Testing	May 17–18, 2026

3. Vulnerability Summary

#	Vulnerability	Category	Status	Severity
1	SQL Injection – Login Bypass	Injection	Confirmed 	Critical
2	SQL Injection – UNION Data Dump	Injection	Confirmed 	Critical
3	OS Command Injection	Injection	Not in Juice Shop	N/A
4	XXE Injection – /etc/passwd Read	Injection	Confirmed 	High
5	Deprecated Interface (Security Misconfiguration)	Misconfiguration	Confirmed 	Medium
6	SSTI – Template Expression Not Evaluated	Injection	Not Vulnerable 	None
7	Brute Force – No Rate Limiting	Authentication	Confirmed 	High
8	Admin Panel Unauthorized Access	Broken Access Control	Confirmed 	Critical
9	Privilege	Broken Access	Confirmed 	Critical

	Escalation via SQL Injection	Control		
10	DOM-Based XSS	Client-Side	Confirmed 	High
11	CORS Misconfiguration	Client-Side	Confirmed 	Medium
12	CSRF – No Token Protection	Client-Side	Confirmed 	Medium

SECTION A — Injection Attacks

A1. SQL Injection — Login Bypass (Login as Admin)

Vulnerability Overview

SQL Injection occurs when untrusted user input is concatenated into a SQL query without proper sanitization or parameterization. An attacker can manipulate the query logic to bypass authentication, extract data, or modify database contents.

Target Location

- URL: `/#/login`
- Vulnerable Field: Email input field

Payload Used

Email field: `' OR 1=1 --`

Password field: anything

How It Works

The single quote (') closes the email string in the SQL query. `OR 1=1` makes the WHERE condition always true, bypassing the need for a valid email. The double dash (--) comments out the rest of the query including the password check. The resulting query becomes: `SELECT * FROM Users WHERE email=' OR 1=1 --' AND password=...` which returns the first user in the database — the administrator.

Findings

Field	Value
Vulnerability	SQL Injection – Authentication Bypass
Endpoint	<code>/rest/user/login</code> (POST)
Result	Logged in as <code>admin@juice-sh.op</code> (User ID: 1)
Severity	Critical
Impact	Full admin access without knowing credentials

Mitigation

- Use parameterized queries / prepared statements
- Implement input validation and whitelist-based filtering
- Never concatenate user input directly into SQL queries

Screenshot Placeholders

 **Screenshot A1a — Burp Suite showing the POST `/rest/user/login` request with the SQL injection payload in the email field**

Payload Used

https://juice-shop.herokuapp.com/rest/products/search?q=qwert'))%20UNION%20SELECT%20id,email,password,'4','5','6','7','8','9'%20FROM%20Users--

Findings

The UNION SELECT statement appended a query against the Users table to the product search query. The response returned a JSON array containing all user records including email addresses and hashed passwords for every registered user.

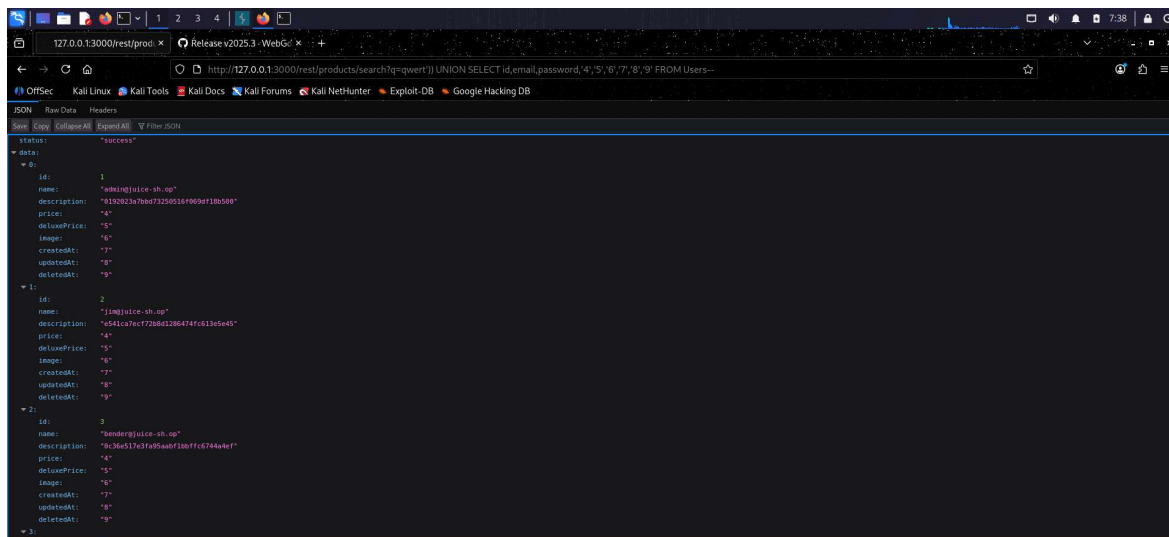
Field	Value
Vulnerability	SQL Injection – UNION Data Extraction
Endpoint	/rest/products/search
Data Extracted	All user emails and hashed passwords
Severity	Critical
Impact	Full database credential dump

Mitigation

- Use ORM frameworks with parameterized queries
- Implement output encoding on JSON responses
- Apply principle of least privilege on database accounts

Screenshot Placeholders

 **Screenshot A2b — JSON response in browser or Burp showing list of all user emails and password hashes**



A4. XML External Entity (XXE) Injection

Vulnerability Overview

XXE Injection exploits vulnerabilities in XML parsers that process external entity references. If the XML parser is configured to allow external entities, an attacker can use a crafted XML file to read local server files, perform SSRF, or cause denial of service.

Target Location

- URL: `/#/complain` (Customer Complaint / Invoice Upload page)
- Backend Endpoint: `POST /file-upload`

Payload Used — `xxe.xml`

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>

<stockCheck>

  <productId>&xxe;</productId>

</stockCheck>
```

Findings

The XML file was uploaded via the complaint page. The server's XML parser processed the external entity declaration and resolved `&xxe;` to the contents of `/etc/passwd`. The file contents were returned embedded within the HTTP 410 error response, confirming the vulnerability. This also simultaneously triggered the Deprecated Interface challenge.

Field	Value
Vulnerability	XXE Injection + Deprecated Interface
Endpoint	<code>POST /file-upload</code>
HTTP Response	410 Gone
Data Extracted	<code>/etc/passwd</code> contents
Users Leaked	root (UID 0), nobody (UID 65534), nonroot (UID 65532)
Severity	High

`/etc/passwd` Contents Extracted

```
root:x:0:0:root:/root:/sbin/nologin
```

```
nobody:x:65534:65534:nobody:/nonexistent:/sbin/nologin
```

```
nonroot:x:65532:65532:nonroot:/home/nonroot:/sbin/nologin
```

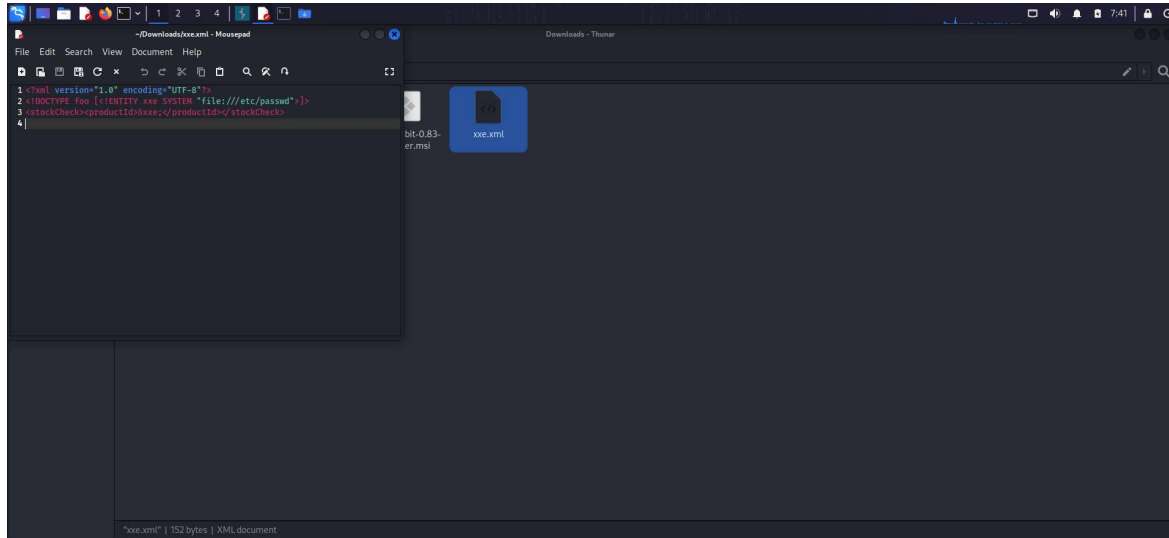
Mitigation

- Disable external entity processing in the XML parser configuration
- Use less complex data formats (JSON) where XML is not required

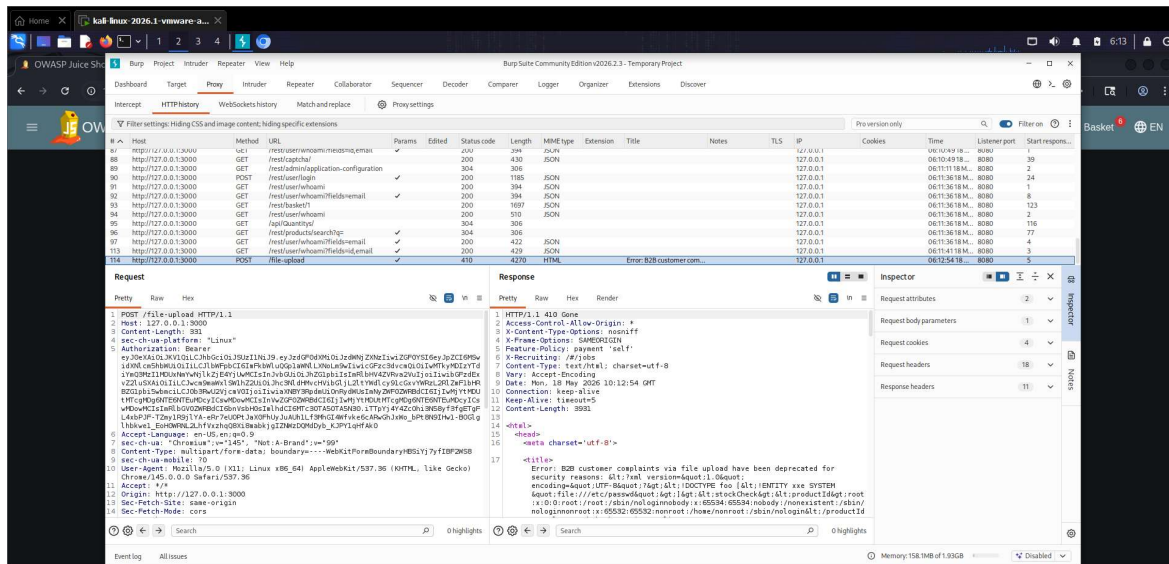
- Properly decommission deprecated API endpoints

Screenshot Placeholders

Screenshot A4a — Contents of xxe.xml payload file in text editor on Kali



Screenshot A4c — Burp Suite response showing 410 Gone with /etc/passwd contents in the HTML title



A5. Server-Side Template Injection (SSTI)

Vulnerability Overview

SSTI occurs when a server-side template engine evaluates user-controlled input as template code rather than plain data. If exploited, an attacker can execute arbitrary expressions or code within the server context.

Test Performed

- Location: /profile (server-rendered profile page)
- Field: Username field
- Payloads tested: `{6*6}`, `{7*7}`, `{9*9}`

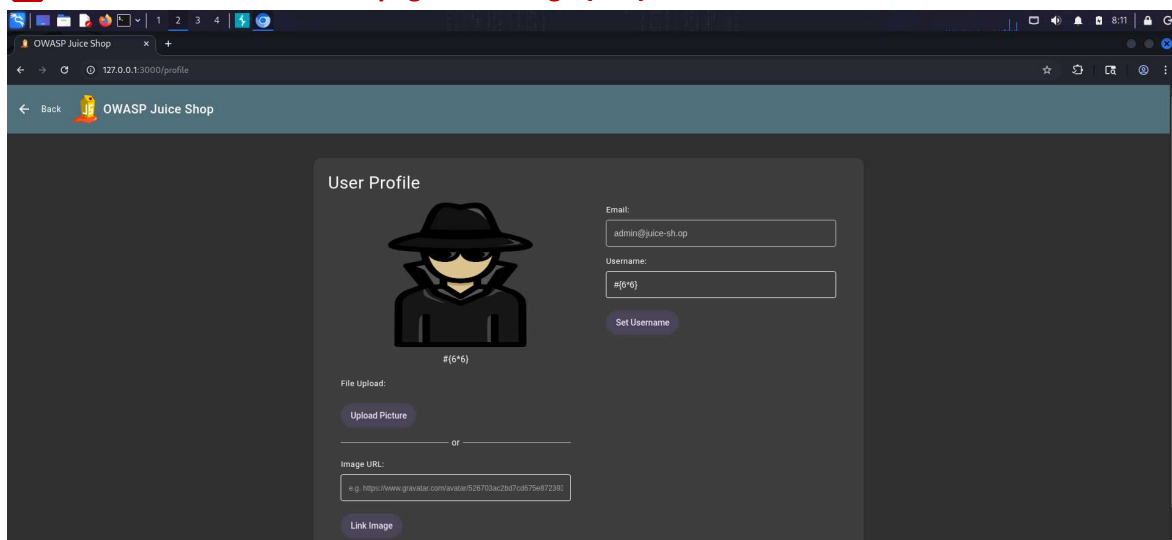
Findings

SSTI test payloads were submitted in the username field of the server-rendered profile page. The JWT token returned in the Set-Cookie header confirmed that the application stored the input as the literal string `"{9*9}"` rather than evaluating it to 81. This indicates that user-controlled data is not interpreted by the server-side Pug/Jade template engine in this field, and no SSTI vulnerability was present in this specific input vector.

Field	Value
Vulnerability	SSTI – Not Vulnerable in This Field
Endpoint	POST /profile
Payload	<code>{9*9}</code>
Expected if Vulnerable	81 rendered as username
Actual Result	<code>{9*9}</code> stored as literal text
Verdict	Input safely handled – no SSTI observed

Screenshot Placeholders

Screenshot A5b — Profile page rendering #{6*6} as literal text instead of 81



SECTION B — Broken Access Control & Authentication

B1. Authentication Vulnerability — Brute Force / No Rate Limiting

Vulnerability Overview

Brute-force attacks involve systematically trying a large number of password combinations against a login endpoint. Applications without rate limiting, account lockout, or CAPTCHA mechanisms are vulnerable to automated credential guessing attacks.

Target Location

- URL: `/#/login`
- Endpoint: `POST /rest/user/login`

Test Performed

Multiple login attempts were made against the `admin@juice-sh.op` account using Burp Suite Intruder with various password combinations. The application was observed to return the same "Invalid email or password" response for every failed attempt without triggering any lockout, delay, or CAPTCHA challenge.

Findings

Field	Value
Vulnerability	No Rate Limiting / No Account Lockout
Endpoint	<code>POST /rest/user/login</code>
Attempts Made	Multiple failed attempts with no lockout observed
CAPTCHA Present	No
Rate Limiting	Not implemented
Account Lockout	Not implemented
Severity	High

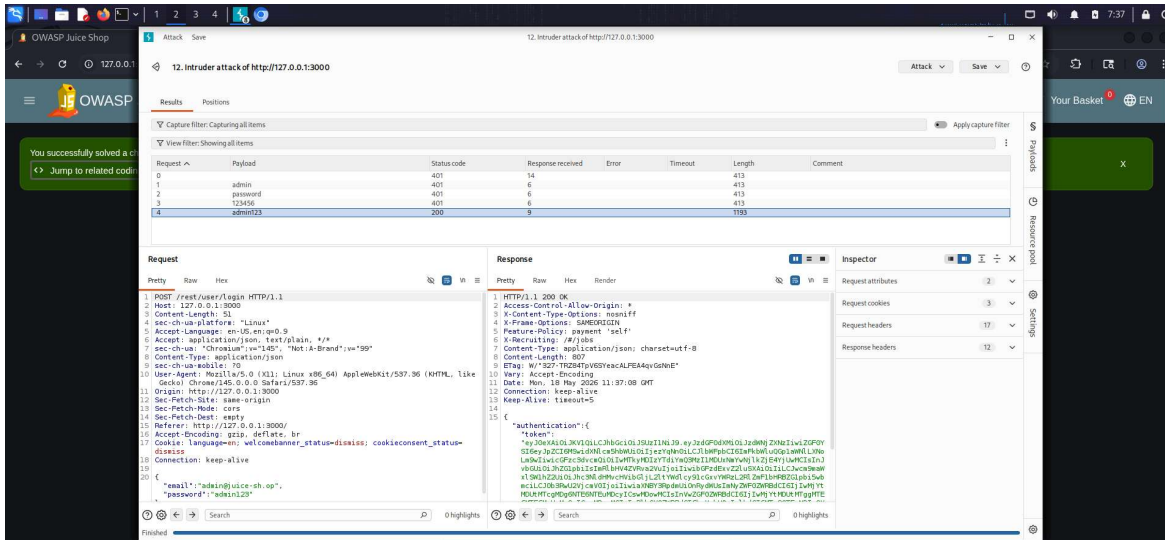
Note: Burp Suite Community Edition throttles Intruder requests due to licensing limitations. In a real engagement, Hydra or ffuf would be used for faster automated brute-force testing.

Mitigation

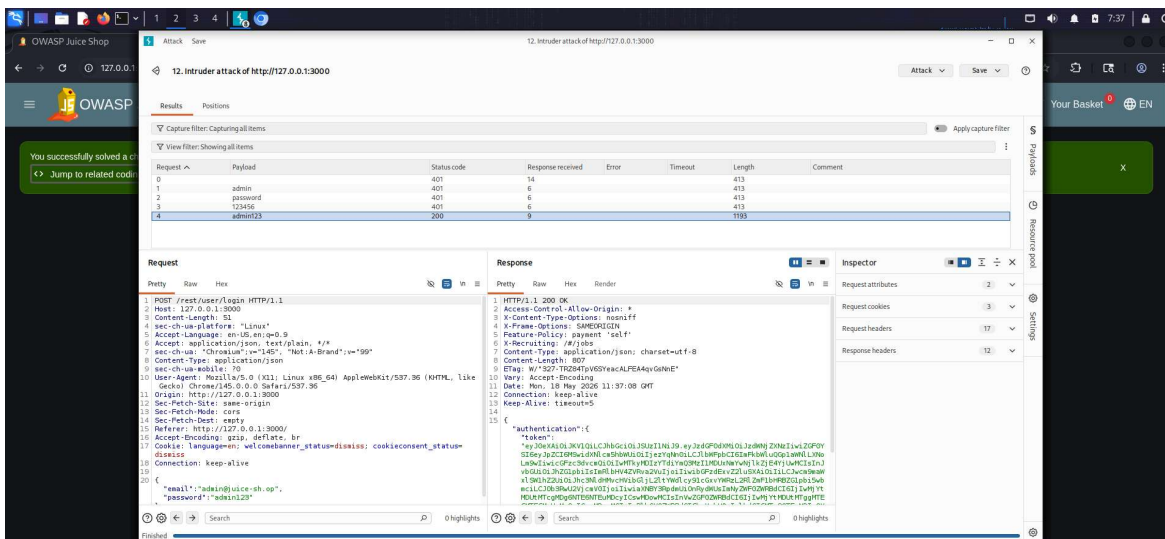
- Implement account lockout after N failed attempts (e.g., 5)
- Add progressive time delays between failed login attempts
- Implement CAPTCHA for repeated failed attempts
- Use multi-factor authentication (MFA/2FA)

Screenshot Placeholders

 **Screenshot B1a — Burp Intruder setup showing multiple password attempts against the login endpoint**



Screenshot B1b — Burp Intruder results showing consistent response for all failed attempts (no lockout)



B2. Authorization Vulnerability — Admin Panel Access

Vulnerability Overview

Authorization vulnerabilities occur when an application fails to properly restrict access to sensitive functions or resources based on the user's role. Forced browsing allows users to navigate directly to URLs they should not have access to.

Test Performed

- Navigated to `/#/administration` without authentication → redirected
- Navigated to `/#/administration` after SQL injection login as admin → full panel accessible

Findings

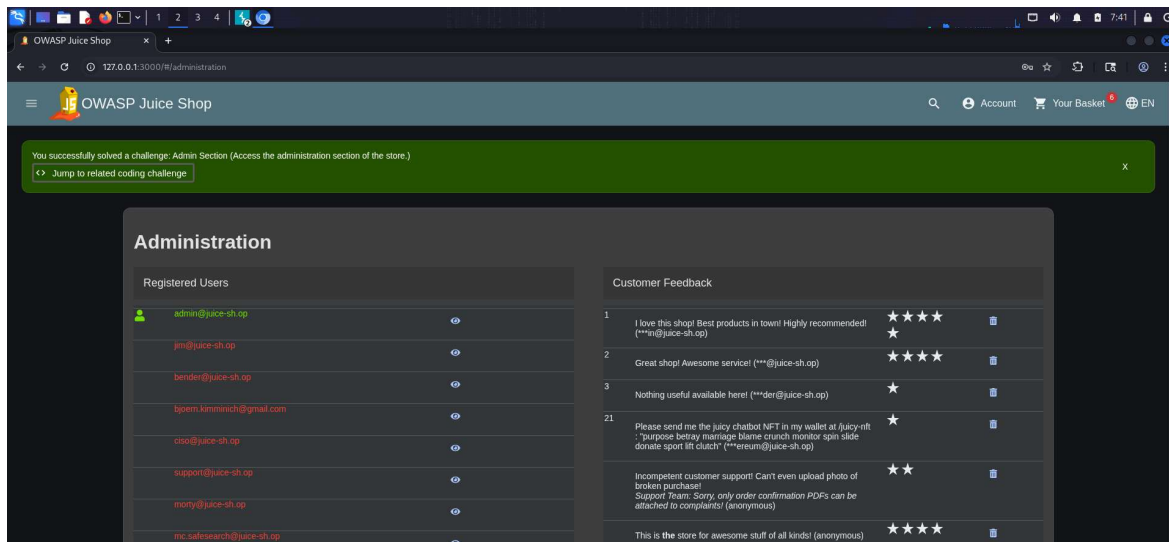
Field	Value
Vulnerability	Insufficient Authorization on Admin Panel
Endpoint	/#/administration
Normal User Access	Redirected / Denied
Admin User Access	Full admin panel accessible
Admin Panel Contents	All users, all feedback, order management
Severity	Critical

Mitigation

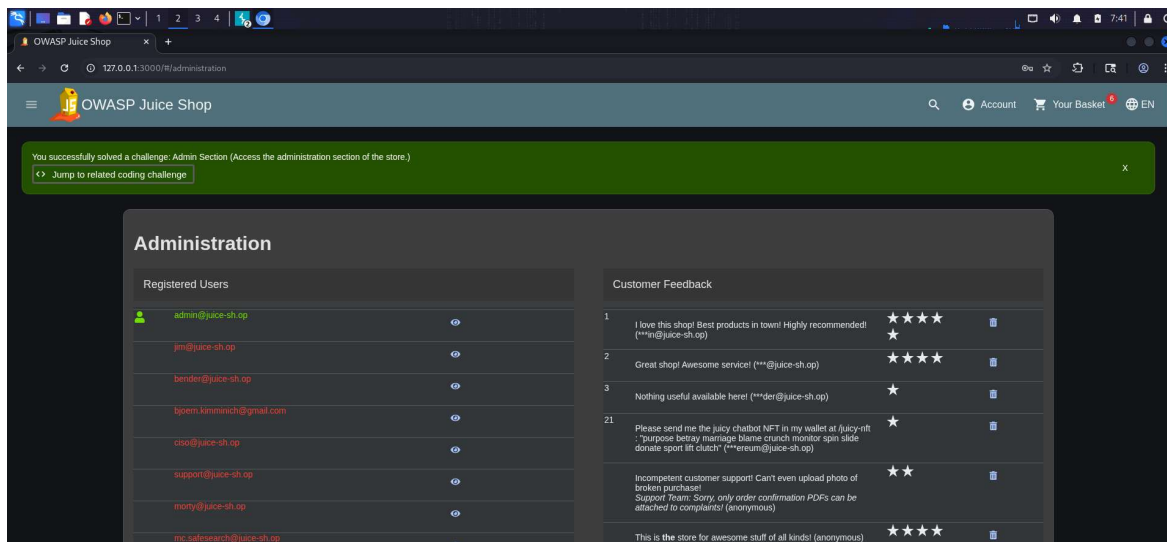
- Enforce server-side role-based access control (RBAC) on all sensitive endpoints
- Never rely solely on client-side routing to restrict access
- Return 403 Forbidden for unauthorized API requests regardless of client navigation

Screenshot Placeholders

 **Screenshot B2a — /#/administration page loaded and showing admin panel as logged-in admin**



 **Screenshot B2b — Admin panel showing full user list and management functions**



B3. Privilege Escalation

Vulnerability Overview

Privilege escalation occurs when a lower-privileged user gains access to higher-privileged functions or data. Vertical privilege escalation involves gaining access to admin-level functionality from a regular user or unauthenticated context.

Test Performed

The `/rest/user/whoami` endpoint was queried before and after SQL injection-based login to compare the session context and role. The JWT token structure was also inspected from Burp response headers.

Evidence — GET `/rest/user/whoami` Response (Admin Session)

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "user": {
    "id": 1,
    "email": "admin@juice-sh.op",
    "lastLoginIp": "",
    "profileImage": "assets/public/images/uploads/defaultAdmin.png"
  }
}
```

Findings

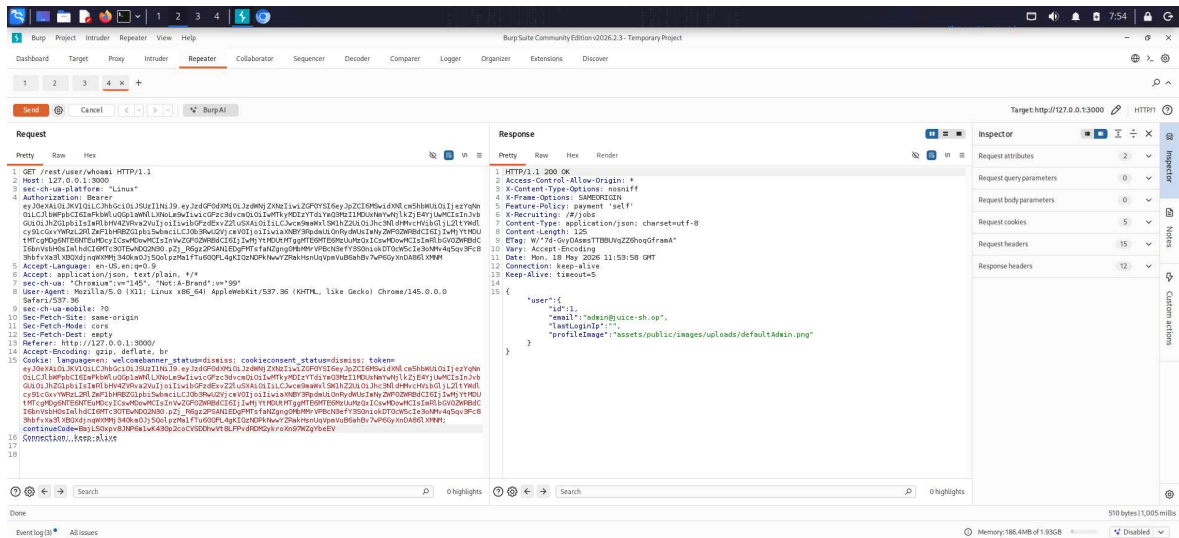
Field	Value
Vulnerability	Vertical Privilege Escalation via SQL Injection
Method	SQL injection on login bypassed authentication entirely
Resulting Privilege	Admin (User ID 1, admin@juice-sh.op)
Admin Capabilities Gained	/#/administration panel, all user data, order management
Severity	Critical

Mitigation

- Parameterized queries eliminate the SQL injection vector used to escalate privileges
- Implement server-side role verification on all privileged endpoints
- Log and alert on anomalous access patterns (e.g., admin panel access from new IPs)

Screenshot Placeholders

 **Screenshot B3a — Burp Repeater showing GET /rest/user/whoami response (200 OK) with admin@juice-sh.op and ID 1**



The screenshot displays the Burp Suite Repeater interface. The 'Request' tab is active, showing a GET request to `/rest/user/whoami` with a status of 200 OK. The response is a JSON object containing user information: `{ "id": 1, "email": "admin@juice-sh.op", "lastLogin": "2020-05-18T11:55:58Z", "profileImage": "assets/public/images/uploads/defaultAdmin.png" }`. The 'Inspector' tab on the right shows the raw response details, including headers and body. The 'Request' tab also shows the raw request details, including headers and body. The 'Inspector' tab is currently showing the 'Request attributes' section.

SECTION C — Client-Side Vulnerabilities

C1. DOM-Based Cross-Site Scripting (XSS)

Vulnerability Overview

DOM-Based XSS occurs when JavaScript reads data from an attacker-controllable source (such as the URL, search parameter, or fragment) and writes it directly into the DOM without sanitization. Unlike reflected XSS, the payload never reaches the server — it is entirely processed and executed on the client side.

Target Location

- URL: `/#/search` (homepage search bar)
- Vulnerable Input: Search query parameter

Payload Used

```
<iframe src="javascript:alert('XSS')">
```

Findings

The search bar accepted and rendered the iframe payload without sanitization. The JavaScript alert was triggered in the browser, confirming that user-controlled input is written directly into the DOM without escaping or encoding. This confirms a DOM-Based XSS vulnerability.

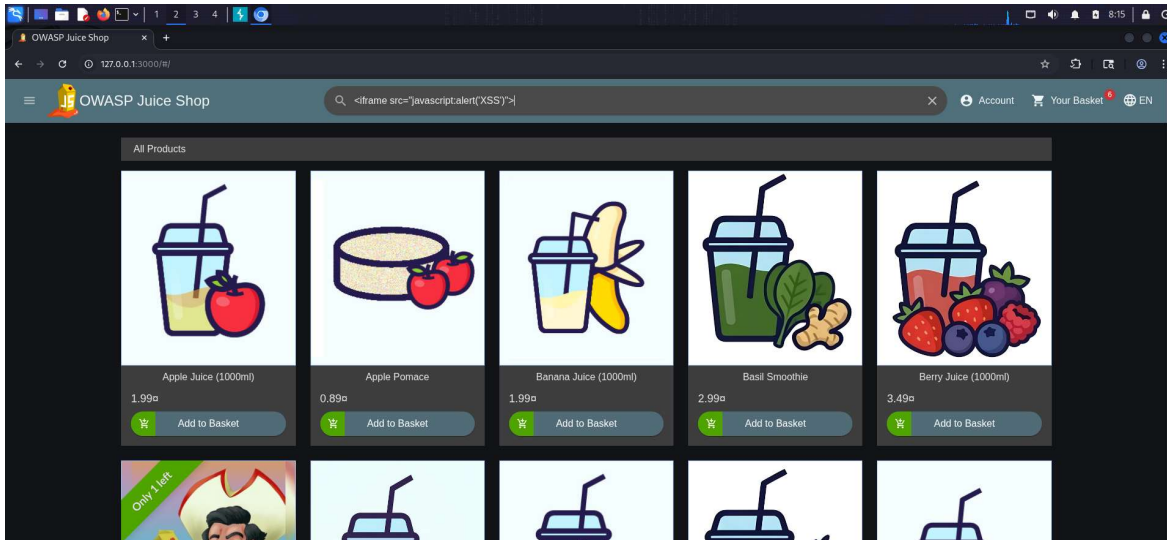
Field	Value
Vulnerability	DOM-Based XSS
Location	Search bar on homepage
Payload	<code><iframe src="javascript:alert('XSS')"></code>
Result	JavaScript alert executed in browser
Severity	High
Impact	Cookie theft, session hijacking, phishing, defacement

Mitigation

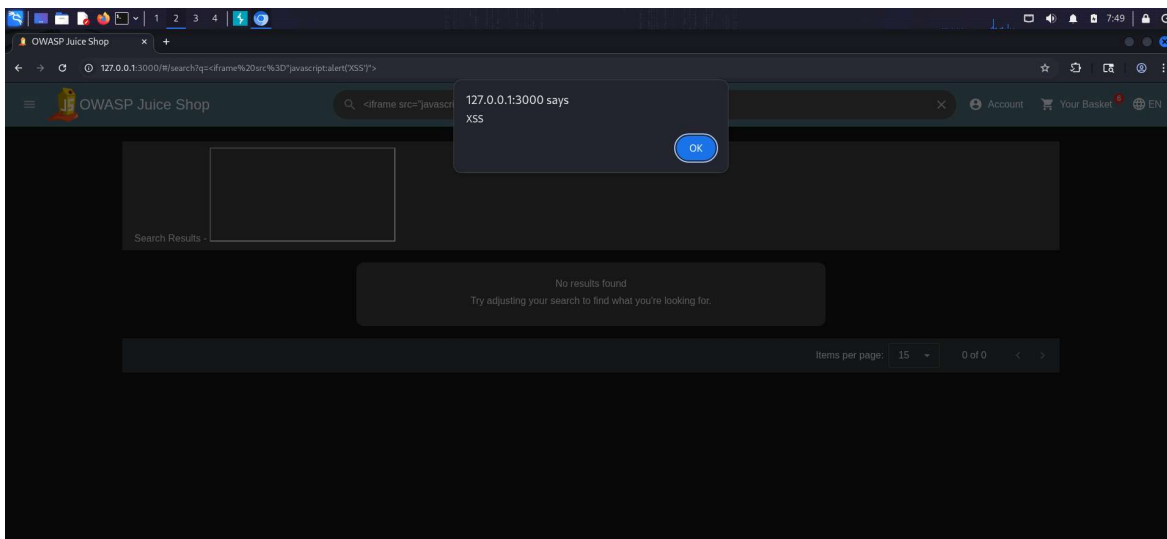
- Sanitize and encode all user input before inserting into the DOM
- Use `textContent` instead of `innerHTML` when inserting untrusted data
- Implement a strong Content-Security-Policy (CSP) header
- Use modern frameworks that auto-escape template expressions

Screenshot Placeholders

 **Screenshot C1a — Search bar with the XSS payload typed in before submission**



Screenshot C1b — Browser alert popup showing the XSS payload executing on the Juice Shop page



C2. Cross-Site Request Forgery (CSRF)

Vulnerability Overview

CSRF tricks an authenticated user's browser into sending unauthorized requests to a web application. Because the browser automatically includes cookies with every request, a malicious page can silently trigger state-changing actions on behalf of the victim.

Test Performed

State-changing requests (profile update, feedback submission) were captured in Burp Suite HTTP history and inspected for the presence of CSRF tokens, custom headers, or other CSRF mitigations.

Findings

Inspection of the profile update and feedback submission requests in Burp Suite revealed no CSRF token, no X-CSRF-Token header, and no unpredictable nonce in the request body. The requests relied solely on the bearer token cookie for authentication, with no additional CSRF protection layer.

Field	Value
Vulnerability	CSRF – No Token Protection
Affected Endpoints	Profile update (POST /profile), Feedback submission
CSRF Token Present	No
Custom Header Required	No
SameSite Cookie Policy	Not enforced
Severity	Medium

Mitigation

- Implement CSRF tokens for all state-changing requests
- Set SameSite=Strict or SameSite=Lax on session cookies
- Use the Synchronizer Token Pattern or Double Submit Cookie pattern

Screenshot Placeholders

 **Screenshot C2a — Burp HTTP History showing a state-changing POST request (e.g., profile update) with NO CSRF token in the request body or headers**

[Attach Screenshot Here]

C3. CORS Misconfiguration

Vulnerability Overview

Cross-Origin Resource Sharing (CORS) misconfiguration occurs when a server's Access-Control-Allow-Origin header is overly permissive, allowing any external website to make cross-origin requests and read the responses. This can enable data theft when combined with authenticated sessions.

Evidence Observed

The Access-Control-Allow-Origin: * header was observed on every API response captured during testing, including /rest/user/whoami, the file upload endpoint, and all other tested endpoints.

Evidence — Observed in /rest/user/whoami Response

HTTP/1.1 200 OK

Access-Control-Allow-Origin: *

Content-Type: application/json

```
{ "user": { "id": 1, "email": "admin@juice-sh.op", "lastLoginIp": "", "profileImage": "..."} }
```

Findings

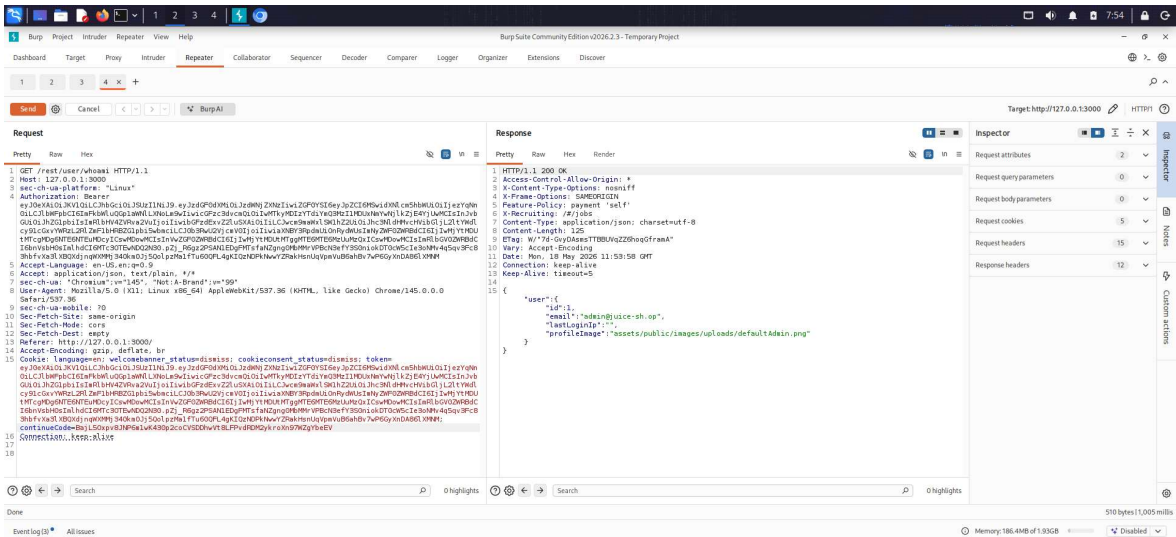
Field	Value
Vulnerability	CORS Misconfiguration
Header Observed	Access-Control-Allow-Origin: *
Affected Endpoints	All tested endpoints including /rest/user/whoami, /file-upload
Credentials Exposed	User email, ID, profile image path on /rest/user/whoami
Severity	Medium
Impact	Cross-origin data theft if combined with XSS or malicious page

Mitigation

- Restrict Access-Control-Allow-Origin to specific trusted domains only
- Never use wildcard (*) with Access-Control-Allow-Credentials: true
- Implement a CORS policy whitelist on the server side

Screenshot Placeholders

 **Screenshot C3a — Burp response headers for GET /rest/user/whoami showing Access-Control-Allow-Origin: ***



4. Conclusion

This vulnerability assessment against the OWASP Juice Shop training platform successfully demonstrated a wide range of web application security vulnerabilities across three major categories: Injection, Broken Access Control & Authentication, and Client-Side attacks.

The most critical findings were the SQL Injection vulnerabilities on the login and search endpoints, which enabled complete authentication bypass and full database credential extraction. The XXE Injection vulnerability allowed server-side file reading (/etc/passwd), while the absence of rate limiting on the login endpoint left it open to brute-force attacks. Client-side vulnerabilities including DOM-Based XSS, CSRF, and CORS misconfiguration further expanded the attack surface.

All findings align with OWASP Top 10 vulnerability categories and demonstrate real-world attack patterns in a safe, authorized training environment. The mitigations documented in each section reflect industry-standard defensive practices that should be applied to production applications.

This report is submitted as part of the Technical Research & Cyber Security Operations Internship at GreySentinel.